

Dags

Apache Airflow 3.3.2

Origem: <https://airflow.apache.org/docs/apache-airflow/3.3.2/core-concepts/dags.html>

Copia de estudo exportada em 21/09/2026. Texto original em ingles; formatacao e paginacao locais. Imagens e navegacao omitidas; consulte a origem para diagramas. Apache License 2.0; creditos em sources/.

Dags

A Dag is a model that encapsulates everything needed to execute a workflow. Some Dag attributes include the following:

Schedule : When the workflow should run.

Tasks : tasks are discrete units of work that are run on workers.

Task Dependencies : The order and conditions under which tasks execute.

Callbacks : Actions to take when the entire workflow completes.

Additional Parameters : And many other operational details.

Here's a basic example Dag:

It defines four Tasks - A, B, C, and D - and dictates the order in which they have to run, and which tasks depend on what others. It will also say how often to run the Dag - maybe "every 5 minutes starting tomorrow", or "every day since January 1st, 2020".

The Dag itself doesn't care about what is happening inside the tasks; it is merely concerned with how to execute them - the order to run them in, how many times to retry them, if they have timeouts, and so on.

Note

The term "DAG" comes from the mathematical concept "directed acyclic graph", but the meaning in Airflow has evolved well beyond just the literal data structure associated with the mathematical DAG concept. Therefore it was decided to use the term Dag in Airflow.

Declaring a Dag

There are three ways to declare a Dag - either you can use with statement (context manager), which will add anything inside it to the Dag implicitly:

```
import datetime

from airflow.sdk import DAG
from airflow.providers.standard.operators.empty import EmptyOperator

with DAG(
    dag_id="my_dag_name",
    start_date=datetime.datetime(2021, 1, 1),
    schedule="@daily",
):
    EmptyOperator(task_id="task")
```

Or, you can use a standard constructor, passing the Dag into any operators you use:

```
import datetime

from airflow.sdk import DAG
from airflow.providers.standard.operators.empty import EmptyOperator

my_dag = DAG(
```

```

    dag_id="my_dag_name",
    start_date=datetime.datetime(2021, 1, 1),
    schedule="@daily",
)
EmptyOperator(task_id="task", dag=my_dag)

```

Or, you can use the @dag decorator to turn a function into a Dag generator :

```

import datetime

from airflow.sdk import dag
from airflow.providers.standard.operators.empty import EmptyOperator

@dag(start_date=datetime.datetime(2021, 1, 1), schedule="@daily")
def generate_dag():
    EmptyOperator(task_id="task")

generate_dag()

```

Dags are nothing without Tasks to run, and those will usually come in the form of either Operators , Sensors or TaskFlow .

Task Dependencies

A Task/Operator does not usually live alone; it has dependencies on other tasks (those upstream of it), and other tasks depend on it (those downstream of it). Declaring these dependencies between tasks is what makes up the Dag structure.

There are two main ways to declare individual task dependencies. The recommended one is to use the >> and << operators:

```

first_task >> [second_task, third_task]
third_task << fourth_task

```

Or, you can also use the more explicit set_upstream and set_downstream methods:

```

first_task.set_downstream([second_task, third_task])
third_task.set_upstream(fourth_task)

```

There are also shortcuts to declaring more complex dependencies. If you want to make a list of tasks depend on another list of tasks, you can't use either of the approaches above, so you need to use cross_downstream :

```

from airflow.sdk import cross_downstream

# Replaces
# [op1, op2] >> op3
# [op1, op2] >> op4
cross_downstream([op1, op2], [op3, op4])

```

And if you want to chain together dependencies, you can use chain :

```

from airflow.sdk import chain

# Replaces op1 >> op2 >> op3 >> op4
chain(op1, op2, op3, op4)

# You can also do it dynamically
chain(*[EmptyOperator(task_id=f"op{i}") for i in range(1, 6)])

```

Chain can also do pairwise dependencies for lists the same size (this is different from the cross dependencies created by cross_downstream !):

```

from airflow.sdk import chain

# Replaces
# op1 >> op2 >> op4 >> op6
# op1 >> op3 >> op5 >> op6
chain(op1, [op2, op3], [op4, op5], op6)

```

Loading Dags

Airflow loads Dags from Python source files in Dag bundles. It will take each file, execute it, and then load any Dag objects from that file.

This means you can define multiple Dags per Python file, or even spread one very complex Dag across multiple Python files using imports.

Note, though, that when Airflow comes to load Dags from a Python file, it will only pull any objects at the top level that are a Dag instance. For example, take this Dag file:

```
dag_1 = DAG('this_dag_will_be_discovered')

def my_function():
    dag_2 = DAG('but_this_dag_will_not')

my_function()
```

While both Dag constructors get called when the file is accessed, only dag_1 is at the top level (in the globals()), and so only it is added to Airflow. dag_2 is not loaded.

Note

When searching for Dags inside the Dag bundle, Airflow only considers Python files that contain the strings airflow and dag (case-insensitively) as an optimization.

To consider all Python files instead, set the [core] dag_discovery_safe_mode configuration flag to False . This is the setting to use when your Dags are defined through a wrapper or abstraction whose source does not contain those strings. The flag is read by the Dag file processor, so set it on that component (and on anything that runs airflow dags reserialize) and restart it for the change to take effect – otherwise Dags may appear after a manual reserialize and then disappear again on the next processor scan.

You can also provide an .airflowignore file inside your Dag bundle, or any of its subfolders, which describes patterns of files for the loader to ignore. It covers the directory it's in plus all subfolders underneath it. See .airflowignore below for details of the file syntax.

In the case where the .airflowignore does not meet your needs and you want a more flexible way to control if a python file needs to be parsed by airflow, you can plug your callable by setting might_contain_dag_callable in the config file. Note, this callable will replace the default Airflow heuristic, i.e. checking if the strings airflow and dag (case-insensitively) are present in the python file.

```
def might_contain_dag(file_path: str, zip_file: zipfile.ZipFile | None = None) -> bool:
    # Your logic to check if there are Dags defined in the file_path
    # Return True if the file_path needs to be parsed, otherwise False
```

Running Dags

Dags will run in one of two ways:

When they are triggered either manually or via the API

On a defined schedule , which is defined as part of the Dag

Dags do not require a schedule, but it's very common to define one. You define it via the schedule argument, like this:

```
with DAG("my_daily_dag", schedule="@daily"):
    ...
```

There are various valid values for the schedule argument:

```
with DAG("my_daily_dag", schedule="0 0 * * *"):
    ...

with DAG("my_one_time_dag", schedule="@once"):
    ...
```

```
with DAG("my_continuous_dag", schedule="@continuous"):
    ...
```

Tip

For more information different types of scheduling, see [Authoring and Scheduling](#) .

Every time you run a Dag, you are creating a new instance of that Dag which Airflow calls a Dag Run . Dag Runs can run in parallel for the same Dag, and each has a defined data interval, which identifies the period of data the tasks should operate on.

As an example of why this is useful, consider writing a Dag that processes a daily set of experimental data. It's been rewritten, and you want to run it on the previous 3 months of data—no problem, since Airflow can backfill the Dag and run copies of it for every day in those previous 3 months, all at once.

Those Dag Runs will all have been started on the same actual day, but each Dag Run will have one data interval covering a single day in that 3 month period, and that data interval is all the tasks, operators and sensors inside the Dag look at when they run.

In much the same way a Dag instantiates into a Dag Run every time it's run, Tasks specified inside a Dag are also instantiated into Task Instances along with it.

A Dag run will have a start date when it starts, and end date when it ends. This period describes the time when the Dag actually 'ran.' Aside from the Dag run's start and end date, there is another date called logical date (formally known as execution date), which describes the intended time a Dag run is scheduled or triggered. The reason why this is called logical is because of the abstract nature of it having multiple meanings, depending on the context of the Dag run itself.

For example, if a Dag run is manually triggered by the user, its logical date would be the date and time of which the Dag run was triggered, and the value should be equal to Dag run's start date. However, when the Dag is being automatically scheduled, with certain schedule interval put in place, the logical date is going to indicate the time at which it marks the start of the data interval, where the Dag run's start date would then be the logical date + scheduled interval.

Tip

For more information on logical date , see [Data Interval and What does execution_date mean?](#) .

Dag Assignment

Note that every single Operator/Task must be assigned to a Dag in order to run. Airflow has several ways of calculating the Dag without you passing it explicitly:

If you declare your Operator inside a with DAG block

If you declare your Operator inside a @dag decorator

If you put your Operator upstream or downstream of an Operator that has a Dag

Otherwise, you must pass it into each Operator with dag= .

Default Arguments

Often, many Operators inside a Dag need the same set of default arguments (such as their retries). Rather than having to specify this individually for every Operator, you can instead pass default_args to the Dag when you create it, and it will auto-apply them to any operator tied to it:

```
import pendulum

with DAG(
    dag_id="my_dag",
    start_date=pendulum.datetime(2016, 1, 1),
    schedule="@daily",
```

```

    default_args={"retries": 2},
):
    op = BashOperator(task_id="hello_world", bash_command="Hello World!")
    print(op.retries) # 2

```

The Dag decorator

Added in version 2.0.

As well as the more traditional ways of declaring a single Dag using a context manager or the Dag() constructor, you can also decorate a function with @dag to turn it into a Dag generator function:

[airflow/example_dags/example_dag_decorator.py \[source\]](#)

```

from typing import TYPE_CHECKING, Any

import httpx
import pendulum

from airflow.providers.standard.operators.bash import BashOperator
from airflow.sdk import BaseOperator, dag, task

if TYPE_CHECKING:
    from airflow.sdk import Context

class GetRequestOperator(BaseOperator):
    """Custom operator to send GET request to provided url"""

    template_fields = ("url",)

    def __init__(self, *, url: str, **kwargs):
        super().__init__(**kwargs)
        self.url = url

    def execute(self, context: Context):
        return httpx.get(self.url).json()

@dag(
    schedule=None,
    start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
    catchup=False,
    tags=["example"],
)
def example_dag_decorator(url: str = "https://httpbingo.org/get"):
    """
    DAG to get IP address and echo it via BashOperator.

    :param url: URL to get IP address from. Defaults to "https://httpbingo.org/get".
    """
    get_ip = GetRequestOperator(task_id="get_ip", url=url)

    @task(multiple_outputs=True)
    def prepare_command(raw_json: dict[str, Any]) -> dict[str, str]:
        external_ip = raw_json["origin"]
        try:
            ipaddress.ip_address(external_ip)
            return {
                "command": f"echo 'Seems like today your server executing Airflow is connected from IP {external_ip}'",
            }
        except ValueError:
            raise ValueError(f"Invalid IP address: '{external_ip}'.")

    command_info = prepare_command(get_ip.output)

    BashOperator(task_id="echo_ip_info", bash_command=command_info["command"])

example_dag = example_dag_decorator()

```

As well as being a new way of making Dags cleanly, the decorator also sets up any parameters you have in your function as Dag parameters, letting you set those parameters when triggering the Dag . You can then access the parameters from Python code, or from `{{ context.params }}` inside a Jinja template .

Note

Airflow will only load Dags that appear in the top level of a Dag file. This means you cannot just declare a function with `@dag` - you must also call it at least once in your Dag file and assign it to a top-level object, as you can see in the example above.

Control Flow

By default, a Dag will only run a Task when all the Tasks it depends on are successful. There are several ways of modifying this, however:

Branching - select which Task to move onto based on a condition

Trigger Rules - set the conditions under which a Dag will run a task

Setup and Teardown - define setup and teardown relationships

Latest Only - a special form of branching that only runs on Dags running against the present

Depends On Past - tasks can depend on themselves from a previous run

Branching

You can make use of branching in order to tell the Dag not to run all dependent tasks, but instead to pick and choose one or more paths to go down. This is where the `@task.branch` decorator come in.

The `@task.branch` decorator is much like `@task` , except that it expects the decorated function to return an ID to a task (or a list of IDs). The specified task is followed, while all other paths are skipped. It can also return `None` to skip all downstream tasks.

The `task_id` returned by the Python function has to reference a task directly downstream from the `@task.branch` decorated task.

Note

When a Task is downstream of both the branching operator and downstream of one or more of the selected tasks, it will not be skipped:

The paths of the branching task are `branch_a` , `join` and `branch_b` . Since `join` is a downstream task of `branch_a` , it will still be run, even though it was not returned as part of the branch decision.

The `@task.branch` can also be used with XComs allowing branching context to dynamically decide what branch to follow based on upstream tasks. For example:

```
@task.branch(task_id="branch_task")
def branch_func(ti=None):
    xcom_value = int(ti.xcom_pull(task_ids="start_task"))
    if xcom_value >= 5:
        return "continue_task"
    elif xcom_value >= 3:
        return "stop_task"
    else:
        return None

start_op = BashOperator(
    task_id="start_task",
    bash_command="echo 5",
    do_xcom_push=True,
    dag=dag,
)
```

```
branch_op = branch_func()

continue_op = EmptyOperator(task_id="continue_task", dag=dag)
stop_op = EmptyOperator(task_id="stop_task", dag=dag)

start_op >> branch_op >> [continue_op, stop_op]
```

If you wish to implement your own operators with branching functionality, you can inherit from `BaseBranchOperator`, which behaves similarly to `@task.branch` decorator but expects you to provide an implementation of the method `choose_branch`.

Note

The `@task.branch` decorator is the Taskflow equivalent of `BranchPythonOperator`. The latter should generally only be subclassed to implement a custom operator.

As with the callable for `@task.branch`, this method can return the ID of a downstream task, or a list of task IDs, which will be run, and all others will be skipped. It can also return `None` to skip all downstream task:

```
class MyBranchOperator(BaseBranchOperator):
    def choose_branch(self, context):
        """
        Run an extra branch on the first day of the month
        """
        if context['data_interval_start'].day == 1:
            return ['daily_task_id', 'monthly_task_id']
        elif context['data_interval_start'].day == 2:
            return 'daily_task_id'
        else:
            return None
```

Similar like `@task.branch` decorator for regular Python code there are also branch decorators which use a virtual environment called `@task.branch_virtualenv` or external python called `@task.branch_external_python`.

Latest Only

Airflow's Dag Runs are often run for a date that is not the same as the current date - for example, running one copy of a Dag for every day in the last month to backfill some data.

There are situations, though, where you don't want to let some (or all) parts of a Dag run for a previous date; in this case, you can use the `LatestOnlyOperator`.

This special Operator skips all tasks downstream of itself if you are not on the "latest" Dag run (if the wall-clock time right now is between its `execution_time` and the next scheduled `execution_time`, and it was not an externally-triggered run).

Here's an example:

[airflow/example_dags/example_latest_only_with_trigger.py](#) [source]

```
import datetime

import pendulum

from airflow.providers.standard.operators.empty import EmptyOperator
from airflow.providers.standard.operators.latest_only import LatestOnlyOperator
from airflow.sdk import DAG, TriggerRule

with DAG(
    dag_id="latest_only_with_trigger",
    schedule=datetime.timedelta(hours=4),
    start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
    catchup=False,
    tags=["example", "example3"],
) as dag:
    latest_only = LatestOnlyOperator(task_id="latest_only")
    task1 = EmptyOperator(task_id="task1")
    task2 = EmptyOperator(task_id="task2")
    task3 = EmptyOperator(task_id="task3")
```

```
task4 = EmptyOperator(task_id="task4", trigger_rule=TriggerRule.ALL_DONE)

latest_only >> task1 >> [task3, task4]
task2 >> [task3, task4]
```

In the case of this Dag:

task1 is directly downstream of latest_only and will be skipped for all runs except the latest.

task2 is entirely independent of latest_only and will run in all scheduled periods

task3 is downstream of task1 and task2 and because of the default trigger rule being all_success will receive a cascaded skip from task1 .

task4 is downstream of task1 and task2 , but it will not be skipped, since its trigger_rule is set to all_done .

Depends On Past

You can also say a task can only run if the previous run of the task in the previous Dag Run succeeded. To use this, you just need to set the depends_on_past argument on your Task to True .

Note that if you are running the Dag at the very start of its life—specifically, its first ever automated run—then the Task will still run, as there is no previous run to depend on.

Trigger Rules

By default, Airflow will wait for all upstream (direct parents) tasks for a task to be successful before it runs that task.

However, this is just the default behaviour, and you can control it using the trigger_rule argument to a Task. The options for trigger_rule are:

all_success (default): All upstream tasks have succeeded

all_failed : All upstream tasks are in a failed or upstream_failed state

all_done : All upstream tasks are done with their execution

all_done_setup_success : Like all_done , but if the task has upstream setup tasks, at least one of them must have succeeded. This is the default trigger rule for teardown tasks.

all_done_min_one_success : All non-skipped upstream tasks are done with their execution and at least one upstream task has succeeded

all_skipped : All upstream tasks are in a skipped state

one_failed : At least one upstream task has failed (does not wait for all upstream tasks to be done)

one_success : At least one upstream task has succeeded (does not wait for all upstream tasks to be done)

one_done : At least one upstream task succeeded or failed

none_failed : All upstream tasks have not failed or upstream_failed - that is, all upstream tasks have succeeded or been skipped

none_failed_min_one_success : All upstream tasks have not failed or upstream_failed , and at least one upstream task has succeeded.

none_skipped : No upstream task is in a skipped state - that is, all upstream tasks are in a success , failed , upstream_failed , or removed state

always : No dependencies at all, run this task at any time

Note

The removed Task Instance state means a task has vanished from the Dag since the run started. For trigger rules, removed is a terminal state: it counts toward “done” for rules like `all_done`, `all_done_setup_success`, and `all_done_min_one_success`, but it does not count as success, failed, upstream_failed, or skipped. For dynamically mapped tasks, removed upstream map indexes are also subtracted from the failure counts of `all_success`, `all_failed`, `none_failed`, `none_failed_min_one_success`, and `all_done_min_one_success`.

You can also combine this with the Depends On Past functionality if you wish.

Note

It's important to be aware of the interaction between trigger rules and skipped tasks, especially tasks that are skipped as part of a branching operation. You almost never want to use `all_success` or `all_failed` downstream of a branching operation.

Skipped tasks will cascade through trigger rules `all_success` and `all_failed`, and cause them to skip as well. Consider the following Dag:

```
# dags/branch_without_trigger.py
import pendulum

from airflow.sdk import task
from airflow.sdk import DAG
from airflow.providers.standard.operators.empty import EmptyOperator

dag = DAG(
    dag_id="branch_without_trigger",
    schedule="@once",
    start_date=pendulum.datetime(2019, 2, 28, tz="UTC"),
)

run_this_first = EmptyOperator(task_id="run_this_first", dag=dag)

@task.branch(task_id="branching")
def do_branching():
    return "branch_a"

branching = do_branching()

branch_a = EmptyOperator(task_id="branch_a", dag=dag)
follow_branch_a = EmptyOperator(task_id="follow_branch_a", dag=dag)

branch_false = EmptyOperator(task_id="branch_false", dag=dag)

join = EmptyOperator(task_id="join", dag=dag)

run_this_first >> branching
branching >> branch_a >> follow_branch_a >> join
branching >> branch_false >> join
```

`join` is downstream of `follow_branch_a` and `branch_false`. The join task will show up as skipped because its trigger_rule is set to `all_success` by default, and the skip caused by the branching operation cascades down to skip a task marked as `all_success`.

By setting trigger_rule to `none_failed_min_one_success` in the join task, we can instead get the intended behaviour:

Setup and teardown

In data workflows it's common to create a resource (such as a compute resource), use it to do some work, and then tear it down. Airflow provides setup and teardown tasks to support this need.

Please see main article Setup and Teardown for details on how to use this feature.

Dynamic Dags

Since a Dag is defined by Python code, there is no need for it to be purely declarative; you are free to use loops, functions, and more to define your Dag.

For example, here is a Dag that uses a for loop to define some tasks:

```
with DAG("loop_example", ...):
    first = EmptyOperator(task_id="first")
    last = EmptyOperator(task_id="last")

    options = ["branch_a", "branch_b", "branch_c", "branch_d"]
    for option in options:
        t = EmptyOperator(task_id=option)
        first >> t >> last
```

In general, we advise you to try and keep the topology (the layout) of your Dag tasks relatively stable; dynamic Dags are usually better used for dynamically loading configuration options or changing operator options.

Dag Visualization

If you want to see a visual representation of a Dag, you have two options:

You can load up the Airflow UI, navigate to your Dag, and select “Graph”

You can run `airflow dags show`, which renders it out as an image file

We generally recommend you use the Graph view, as it will also show you the state of all the Task Instances within any Dag Run you select.

Of course, as you develop out your Dags they are going to get increasingly complex, so we provide a few ways to modify these Dag views to make them easier to understand.

TaskGroups

A TaskGroup can be used to organize tasks into hierarchical groups in Graph view. It is useful for creating repeating patterns and cutting down visual clutter.

Tasks in TaskGroups live on the same original Dag, and honor all the Dag settings and pool configurations.

See also

API reference for TaskGroup and `task_group`

Dependency relationships can be applied across all tasks in a TaskGroup with the `>>` and `<<` operators. For example, the following code puts `task1` and `task2` in TaskGroup `group1` and then puts both tasks upstream of `task3`:

```
from airflow.sdk import task_group

@task_group()
def group1():
    task1 = EmptyOperator(task_id="task1")
    task2 = EmptyOperator(task_id="task2")

task3 = EmptyOperator(task_id="task3")

group1() >> task3
```

TaskGroup also supports `default_args` like Dag, it will overwrite the `default_args` in Dag level:

```
import datetime

from airflow.sdk import DAG
from airflow.sdk import task_group
from airflow.providers.standard.operators.bash import BashOperator
from airflow.providers.standard.operators.empty import EmptyOperator

with DAG(
```

```

dag_id="dag1",
start_date=datetime.datetime(2016, 1, 1),
schedule="@daily",
default_args={"retries": 1},
):

@task_group(default_args={"retries": 3})
def group1():
    """This docstring will become the tooltip for the TaskGroup."""
    task1 = EmptyOperator(task_id="task1")
    task2 = BashOperator(task_id="task2", bash_command="echo Hello World!", retries=2)
    print(task1.retries) # 3
    print(task2.retries) # 2

```

If you want to see a more advanced use of TaskGroup, you can look at the `example_task_group_decorator.py` example Dag that comes with Airflow.

Note

By default, child tasks/TaskGroups have their IDs prefixed with the `group_id` of their parent TaskGroup. This helps to ensure uniqueness of `group_id` and `task_id` throughout the Dag.

To disable the prefixing, pass `prefix_group_id=False` when creating the TaskGroup, but note that you will now be responsible for ensuring every single task and group has a unique ID of its own.

Note

When using the `@task_group` decorator, the decorated-function's docstring will be used as the TaskGroups tooltip in the UI except when a tooltip value is explicitly supplied.

Edge Labels

As well as grouping tasks into groups, you can also label the dependency edges between different tasks in the Graph view - this can be especially useful for branching areas of your Dag, so you can label the conditions under which certain branches might run.

To add labels, you can use them directly inline with the `>>` and `<<` operators:

```

from airflow.sdk import Label

my_task >> Label("When empty") >> other_task

```

Or, you can pass a Label object to `set_upstream` / `set_downstream` :

```

from airflow.sdk import Label

my_task.set_downstream(other_task, Label("When empty"))

```

Here's an example Dag which illustrates labeling different branches:

[airflow/example_dags/example_branch_labels.py](#) [source]

```

with DAG(
    "example_branch_labels",
    schedule="@daily",
    start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
    catchup=False,
    tags=["example"],
) as dag:
    ingest = EmptyOperator(task_id="ingest")
    analyse = EmptyOperator(task_id="analyze")
    check = EmptyOperator(task_id="check_integrity")
    describe = EmptyOperator(task_id="describe_integrity")
    error = EmptyOperator(task_id="email_error")
    save = EmptyOperator(task_id="save")
    report = EmptyOperator(task_id="report")

    ingest >> analyse >> check
    check >> Label("No errors") >> save >> report
    check >> Label("Errors found") >> describe >> error >> report

```

Dag, Task Group & Task Documentation

It's possible to add documentation or notes to your Dags, TaskGroups, and task objects that are visible in the web interface.

There are a set of special task attributes that get rendered as rich content if defined:

attribute rendered to

doc monospace

doc_json json

doc_yaml yaml

doc_md markdown

doc_rst reStructuredText

Please note that for Dags and TaskGroups, doc_md is the only attribute interpreted. It can contain a string or the reference to a markdown file. Markdown files are recognized by str ending in .md . If a relative path is supplied it will be loaded from the path relative to which the Airflow Scheduler or Dag parser was started. If the markdown file does not exist, the passed filename will be used as text, no exception will be displayed. Note that the markdown file is loaded during Dag parsing, changes to the markdown content take one Dag parsing cycle to have changes be displayed.

This is especially useful if your tasks are built dynamically from configuration files, as it allows you to expose the configuration that led to the related tasks in Airflow:

```
"""
### My great Dag
"""

import pendulum
from airflow.providers.standard.operators.empty import EmptyOperator
from airflow.sdk import DAG, TaskGroup

with DAG(
    "my_dag",
    start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
    schedule="@daily",
    catchup=False,
) as dag:
    dag.doc_md = __doc__

    t = EmptyOperator(task_id="foo")
    t.doc_md = """\
#Title
Here's a [url](www.airbnb.com)
"""

    with TaskGroup("extract", doc_md="### Extract tasks"):
        EmptyOperator(task_id="extract_orders")
```

Packaging Dags

While simpler Dags are usually only in a single Python file, it is not uncommon that more complex Dags might be spread across multiple files and have dependencies that should be shipped with them ("vendored").

You can either do this all inside of the Dag bundle, with a standard filesystem layout, or you can package the Dag and all of its Python files up as a single zip file. For instance, you could ship two Dags along with a dependency they need as a zip file with the following contents:

```
my_dag1.py
my_dag2.py
package1/__init__.py
package1/functions.py
```

Note that packaged Dags come with some caveats:

They cannot be used if you have pickling enabled for serialization

They cannot contain compiled libraries (e.g. libz.so), only pure Python

They will be inserted into Python's sys.path and importable by any other code in the Airflow process, so ensure the package names don't clash with other packages already installed on your system.

In general, if you have a complex set of compiled dependencies and modules, you are likely better off using the Python virtualenv system and installing the necessary packages on your target systems with pip .

.airflowignore

An .airflowignore file specifies the directories or files in the Dag bundle or PLUGINS_FOLDER that Airflow should intentionally ignore. Airflow supports two syntax flavors for patterns in the file, as specified by the DAG_IGNORE_FILE_SYNTAX configuration parameter (added in Airflow 2.3): regexp and glob .

Note

The default DAG_IGNORE_FILE_SYNTAX is glob in Airflow 3 or later (in previous versions it was regexp).

With the glob syntax (the default), the patterns work just like those in a .gitignore file:

The * character will match any number of characters, except /

The ? character will match any single character, except /

The range notation, e.g. [a-zA-Z] , can be used to match one of the characters in a range

A pattern can be negated by prefixing with ! . Patterns are evaluated in order so a negation can override a previously defined pattern in the same file or patterns defined in a parent directory.

A double asterisk (**) can be used to match across directories. For example, **/__pycache__ / will ignore __pycache__ directories in each sub-directory to infinite depth.

If there is a / at the beginning or middle (or both) of the pattern, then the pattern is relative to the directory level of the particular .airflowignore file itself. Otherwise the pattern may also match at any level below the .airflowignore level.

For the regexp pattern syntax, each line in .airflowignore specifies a regular expression pattern, and directories or files whose names (not Dag id) match any of the patterns would be ignored (under the hood, Pattern.search() is used to match the pattern). Use the # character to indicate a comment; all characters on lines starting with # will be ignored.

The .airflowignore file should be put in your Dag bundle. For example, you can prepare a .airflowignore file with the glob syntax

```
**/*project_a*
tenant_[0-9]*
```

Then files like project_a_dag_1.py , TESTING_project_a.py , tenant_1.py , project_a/dag_1.py , and tenant_1/dag_1.py in your Dag bundle would be ignored (If a directory's name matches any of the patterns, this directory and all its subfolders would not be scanned by Airflow at all. This improves efficiency of Dag finding).

The scope of a .airflowignore file is the directory it is in plus all its subfolders. You can also prepare .airflowignore file for a subfolder in your Dag bundle and it would only be applicable for that subfolder.

Dag Dependencies

Added in Airflow 2.1

While dependencies between tasks in a Dag are explicitly defined through upstream and downstream relationships, dependencies between Dags are a bit more complex. In general, there are two ways in which one Dag can depend on another:

triggering - TriggerDagRunOperator

waiting - ExternalTaskSensor

Additional difficulty is that one Dag could wait for or trigger several runs of the other Dag with different data intervals. These dependencies are calculated by the scheduler during Dag serialization.

The dependency detector is configurable, so you can implement your own logic different than the defaults in DependencyDetector

Dag pausing, deactivation and deletion

The Dags have several states when it comes to being “not running”. Dags can be paused, deactivated and finally all metadata for the Dag can be deleted.

Dag can be paused via UI when it is present in the DAGS_FOLDER , and scheduler stored it in the database, but the user chose to disable it via the UI. The “pause” and “unpause” actions are available via UI and API. Paused Dags are not scheduled by the Scheduler, but you can trigger them via UI for manual runs. In the UI, you can see paused Dags (in Paused tab). The Dags that are un-paused can be found in the Active tab. When a Dag is paused, any running tasks are allowed to complete and all downstream tasks are put in to a state of “Scheduled”. When the Dag is unpaused, any “scheduled” tasks will begin running according to the Dag logic. Dags with no “scheduled” tasks will begin running according to their schedule.

Dags can be deactivated (do not confuse it with Active tag in the UI) by removing them from the DAGS_FOLDER . When scheduler parses the DAGS_FOLDER and misses the Dag that it had seen before and stored in the database it will set is as deactivated. The metadata and history of the Dag is kept for deactivated Dags and when the Dag is re-added to the DAGS_FOLDER it will be again activated and history will be visible. You cannot activate/deactivate Dag via UI or API, this can only be done by removing files from the DAGS_FOLDER . Once again - no data for historical runs of the Dag are lost when it is deactivated by the scheduler. Note that the Active tab in Airflow UI refers to Dags that are not both Activated and Not paused so this might initially be a little confusing.

You can't see the deactivated Dags in the UI - you can sometimes see the historical runs, but when you try to see the information about those you will see the error that the Dag is missing.

You can also delete the Dag metadata from the metadata database using UI or API, but it does not always result in disappearing of the Dag from the UI - which might be also initially a bit confusing. If the Dag is still in DAGS_FOLDER when you delete the metadata, the Dag will re-appear as Scheduler will parse the folder, only historical runs information for the Dag will be removed.

This all means that if you want to actually delete a Dag and its all historical metadata, you need to do it in three steps:

pause the Dag

delete the historical metadata from the database, via UI or API

delete the Dag file from the DAGS_FOLDER and wait until it becomes inactive

Dag Auto-pausing (Experimental)

Dags can be configured to be auto-paused as well. There is a Airflow configuration which allows for automatically disabling of a Dag if it fails for N number of times consecutively.

max_consecutive_failed_dag_runs_per_dag

we can also provide and override these configuration from Dag argument:

`max_consecutive_failed_dag_runs` : Overrides `max_consecutive_failed_dag_runs_per_dag` .

Deadline Alerts

Added in version 3.1.

Deadline Alerts allow you to set time thresholds for your Dag runs and automatically respond when those thresholds are exceeded. You can set deadlines relative to a fixed datetime, use one of the available calculated references (like Dag queue time or start time), or implement your own custom reference. When a deadline is exceeded, it triggers a callback which can notify you or take other actions.

Here's a simple example using the existing email Notifier:

```
from datetime import timedelta
from airflow import DAG
from airflow.providers.smtp.notifications.smtp import SmtplibNotifier
from airflow.sdk.definitions.deadline import DeadlineAlert, DeadlineReference

with DAG(
    dag_id="email_deadline",
    deadline=DeadlineAlert(
        reference=DeadlineReference.DAGRUN_QUEUED_AT,
        interval=timedelta(minutes=30),
        callback=SmtplibNotifier(
            to="team@example.com",
            subject="■ Dag {{ dag_run.dag_id }} missed deadline at {{ deadline.deadline_time }}",
            html_content="The Dag Run {{ dag_run.dag_run_id }} has been running for more than 30 minutes
since being queued.",
        ),
    ),
):
    EmptyOperator(task_id="task1")
```

This example will send an email notification if the Dag hasn't finished 30 minutes after it was queued.

For more information on implementing and configuring Deadline Alerts, see [Deadline Alerts](#) .

Testing a Dag

In order to verify the syntax/functionality of Dag code, there is a handy option available.

Dag objects have a `test()` function, which can be invoked either directly or within a test suite.

Simulated dag run

A simplest way to check validity of a dag is to use the following snippet at the end of the dag definition module:

```
if __name__ == "__main__":
    dag.test()
```

and execute the module as a Python script (assuming sufficient environment such as `AIRFLOW_HOME` is provided).

The `dag.test()` call will invoke a simulated execution flow, similar to a `LocalExecutor` execution session, and run the Dag contents as in an Airflow submission.

Real execution flow

In case you would like to test the Dag against a real Airflow Executor, the same mechanism can be used. Addressing the call with the `use_executor` flag, the Airflow Executor of currently applied Airflow Configuration will be invoked, and run the workloads of the Dag.

```
dag.test(use_executor=True)
```

Using the call within a pytest test suite, you may benefit of the Airflow pytest plugin's `conf_vars` fixture, that allows easy alternation of pre-defined configuration values.